Lab.1

1.

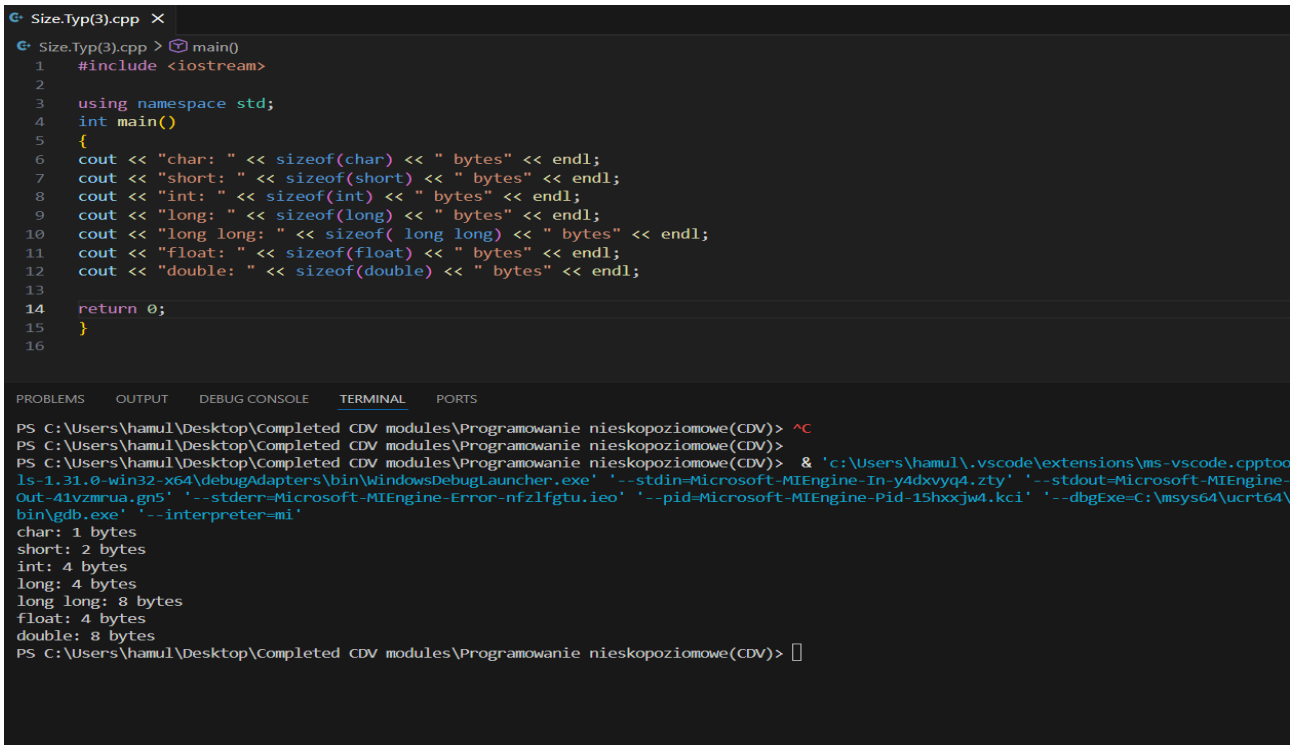
Typy
char – 1 bajt
short – 2 bajty
int – 4 bajt
long – 4 albo 8(zależy od systemu)
long long – 8 bajtów
float – 4 bajty
double – 8 bajty
bool – 1 bajt
String - 24 bajty(W c++, w zwykłej C nie ma typu String).

2.

Int – Służy do przechowywania liczb całkowitych.
Zastosowanie: liczniki pętli indeksy tablic, liczby osób.
Przykład: int wiek = 25;
Float – Typ zmiennoprzecinkowy pojedynczej precyzji. Przechowuje około 6-7 cyfr dziesiętnych.
Zastosowanie: pomiary z czujników, gdzie niewielki błąd nie jest krytyczny.
Przykład: float temperatura = 36.6f; P.S Wymaga przyrostka **F** przy definicji
double – Typ zmiennoprzecinkowy podwójnej precyzji. Przechowuje około 15 cyfr dziesiętnych.
Standart w obliczeniach matematycznych w C.
Zastosowanie Precyzyjne obliczenia inżynierskie, astronomiczne, finansowe.
Przykład: double liczbaPi = 3.1415926535;
Char – Służy do przechowywania pojedynczego znaku ASCII, reprezentowany jako liczba całkowita.
Zastosowanie: Przechowywanie liter, cyfr(jako znaków), lub symboli.
Przykład: char klasa = ‘A’;

3.

Program wykorzystuje operator **sizeof**, aby określić liczbę bajtów zajmowanych w pamięci operacyjnej przez typy danych.



4.

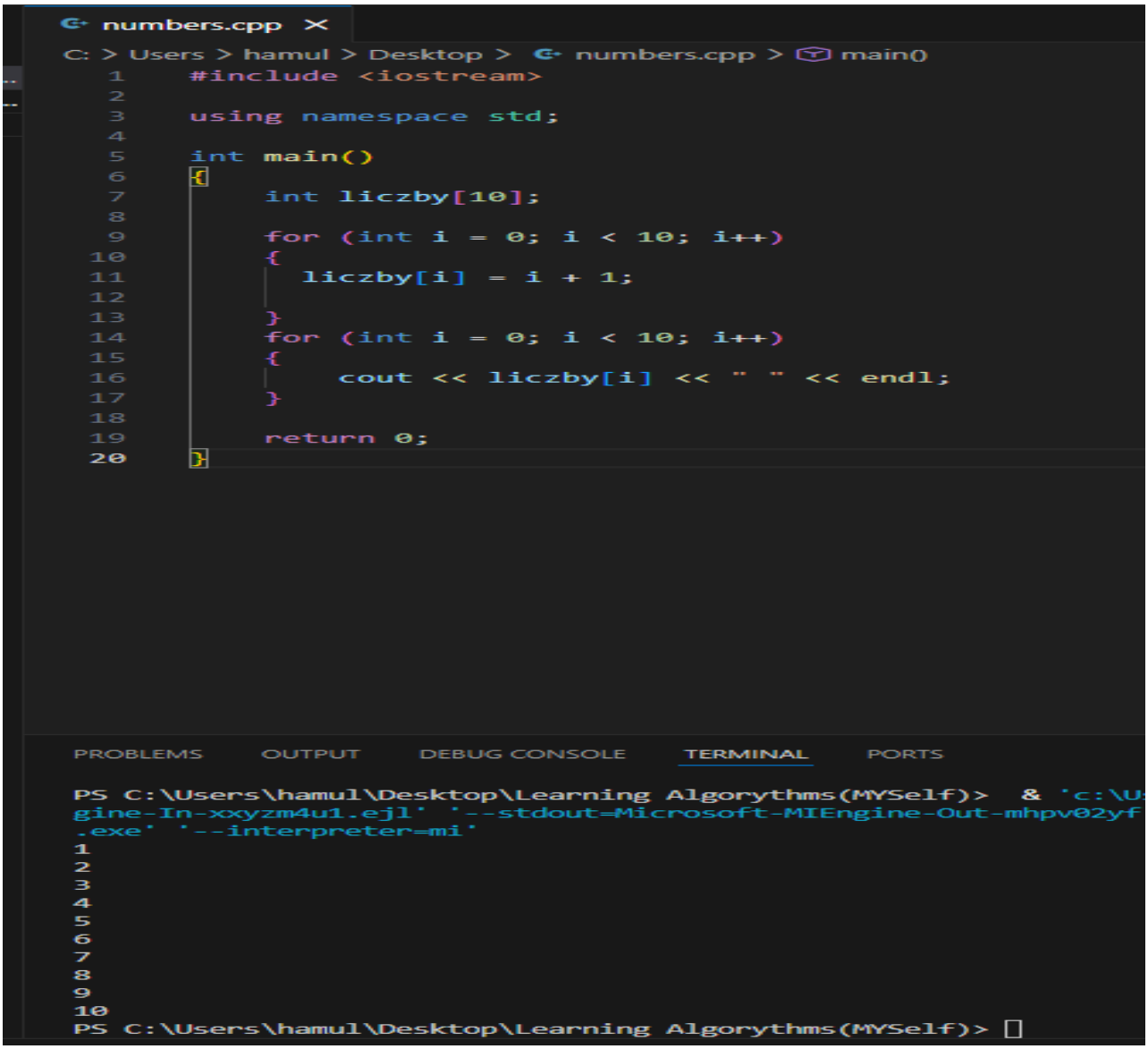
Zakres wartości typu całkowitego określa minimalną i maksymalną liczbę, jaką można przechować w zmiennej tego typu.
`Zakres dla unsigned char:` Typ `unsigned char` przechowuje wyłącznie liczby nieujemne. Dlatego jego zakres wynosi **od 0 do 255**.

5.

Signed – przechowywanie dodatniej i ujemnej liczby.
Unsigned – przechowywanie tylko dodatniej liczby
const – Oznacza, że wartość zmiennej nie może być zmieniona po jej zainicjalizowaniu
volatile - informuje kompilator, że wartość zmiennej może ulec zmianie "bez wiedzy" programu, więc nie należy jej optymalizowac.

6.

Program definiuje 10-elementową tablicę typu int, a następnie wykorzystuje pętle for do wypełnienia jej liczbami od 1 do 10.



7.

int a[10] - Tablica statyczna, kompilator od razu rezerwuje w pamięci miejsce na dokładnie 10 elementów typu int.
int *a – Wskaznik, rezerwuje w pamięci jedynie miejsce na adres, ale nie przydziela pamięci na same liczby.
Brak różnicy: Kiedy wskaźnik zostanie już powiązany z obszarem pamięci, dostęp do elementów w obu przypadkach wygląda tak samo – można używać operatora indeksowania.

8.

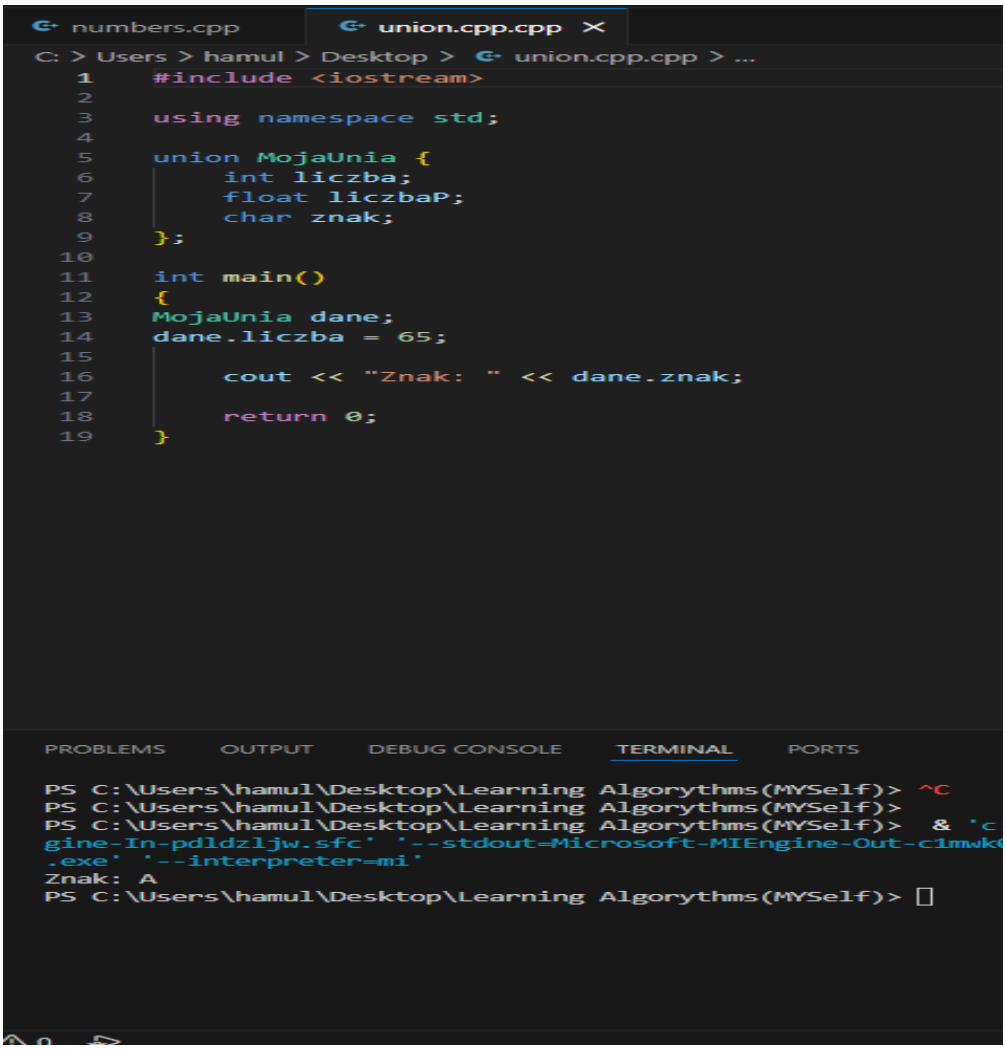
Różnica dotyczy zarządzania pamięcią

STRUCT - Kompilator alokuje osobną pamięć dla każdego pola. Rozmiar struktury to co najmniej suma rozmiarów wszystkich jej elementów.

UNION - Wszystkie pola współdzielą dokładnie ten sam fragment pamięci. Rozmiar unii jest równy rozmiarowi jej największego elementu. W danym momencie można przechowywać wartość tylko w jednym polu – zapisanie danych do nowego pola automatycznie nadpisuje poprzednie.

9.

Ponieważ wszystkie pola w unii współdzzielą ten sam obszar w pamięci, przypisanie wartości **65** do pola **liczba(int),** zapisuje te dane we wspólnym bloku. Kiedy odczytach pola **znak(char)** to kompilator interpretuje te same binarne dziesiętna 65 odpowiada wielkej literze ‘**A’** .Udowadnia to, że zapis do jednego pola unii bezpośrednio wpływa na odczyt z pozostałych, ponieważ modyfikują one ten sam fragment pamięci operacyjnej.



10.

Rozmiar poniższej struktury wynosi 12 bajtów.

Wyjaśnienie: Wynika to z mechanizmu wyrównywania pamięci (memory padding). Suma wielkości samych typów to 6 bajtów (1 + 4 + 1). Jednak procesor optymalizuje dostęp do pamięci, odczytując ją w blokach.
Kompilator dodaje **3 puste bajty** po zmiennej **char c,** aby zmienna **int i** zaczynała się od adresu podzielnego przez 4, dodaje również **3 puste bajty** na końcu po zmiennej **char d,** aby całkowity rozmiar struktury był wielokrotnością, jej największego składnika.
ZATEM 4 + 4 + 4 = 12 bajtów.

11.

Tak, jeżeli zapisać w takiej kołejnośći:
struct A {
int i;
char c;
char d;
};
W tej kolejności zmienne typu `char` zostaną "spakowane" w jednym bloku pamięci, a całkowity rozmiar struktury zmniejszy się do **8 bajtów**.

12.

Porównanie:
Przez wartość: Funkcja robi sobie "ksero" naszej struktury. Cokolwiek w niej zmienimy (np. x = 100), oryginał w `main` pozostaje bez zmian. Jest to bezpieczne, ale przy dużych strukturach kopiowanie niepotrzebnie zżera pamięć.
Przez wskaźnik: Funkcja dostaje tylko "link" (adres) do oryginału. Kiedy używamy strzałki `->` i zmieniamy dane (np. x = 200), oryginał w `main` też się zmienia na stałe. Działa to mega szybko, bo niczego nie kopiujemy.



13.

W zadaniu stworzyłem własny typ danych **Zespolowa** za pomocą **struct** która przehowuje dwie wartości typu **double** część rzeczywistą (**re**) i urojoną (**im**).
Następnie napisałem trzy funkcje: 1.**wyswietl –** formatuje i wypisuje liczbę w czytelnej postaci "a + b i".
2. dodaj i **pomnoz -** przyjmują dwie struktury, wykonują na ich polach odpowiednie operacje matematyczne pakują wynik do nowej struktury **wynik** i ja zwracają.
W funkcji **main** przetestowałem działanie, tworząc dwie przykładowe liczby zespolone i wywołując na nich napisane funkcje.

struct.cpp | strpar.cpp | complexNum.cpp

C: > Users > hamul > Desktop > Completed CDV modules > Programowanie nieskopozior

```cpp
#include <iostream>

using namespace std;
struct Zespolowa
{
    double re, im;
};

void wyswietl(Zespolowa z)
{
    cout << z.re << " + " << z.im << " i" << endl;
}

Zespolowa dodaj(Zespolowa z1, Zespolowa z2)
{
    Zespolowa wynik;

    wynik.re = z1.re + z2.re;
    wynik.im = z1.im + z2.im;

    return wynik;
}
Zespolowa pomnoz(Zespolowa z1, Zespolowa z2)
{
Zespolowa wynik;
    wynik.re = (z1.re * z2.re) - (z1.im * z2.im);
    wynik.im = (z1.re * z2.im) + (z1.im * z2.re);

    return wynik;
}
int main()
{
Zespolowa liczba1 = {2.0, 3.0};
Zespolowa liczba2 = {2.0, 3.0};

Zespolowa suma = dodaj(liczba1,liczba2);
Zespolowa iloczyn = pomnoz(liczba1, liczba2);

wyswietl(suma);
wyswietl(iloczyn);
return 0;


}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
4 + 6 i
-5 + 12 i
PS C:\Users\hamul\Desktop\Learning Algorythms(MYSelf)>
```